23CSE455 Design Patterns

Design Pattern Identification and Evolution in a Quarkus-Based Digital Wallet Application
Case Study Report - Team B9

Case study: Digital Wallet Application using Quarkus Date: 2026-10-11

Students

Team member names, roll numbers, and batches were not provided. This document deliberately does not invent personal details; complete the fields and adjust row count to the official roster before submission.

Roll number	Student name	Batch
To be supplied	To be supplied	To be supplied
To be supplied	To be supplied	To be supplied

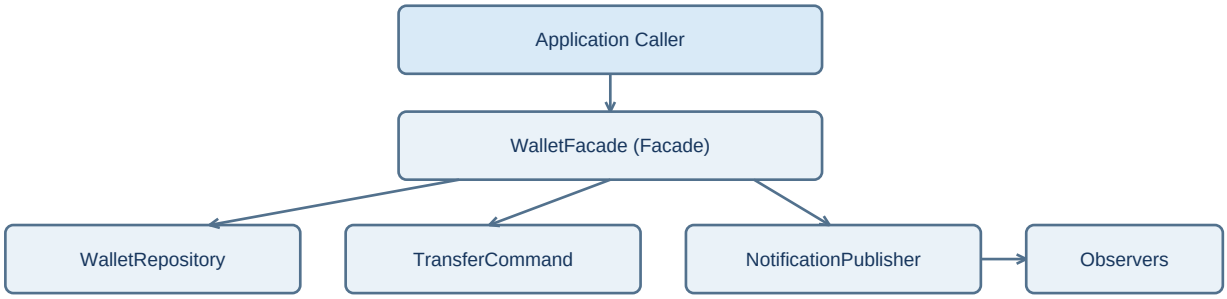
1. Application and architecture

The academic reference implementation models wallet balances, beneficiaries, transfer execution, notification, currency conversion, authentication-provider selection, transaction limits, and fraud screening. It is not a production payment service.

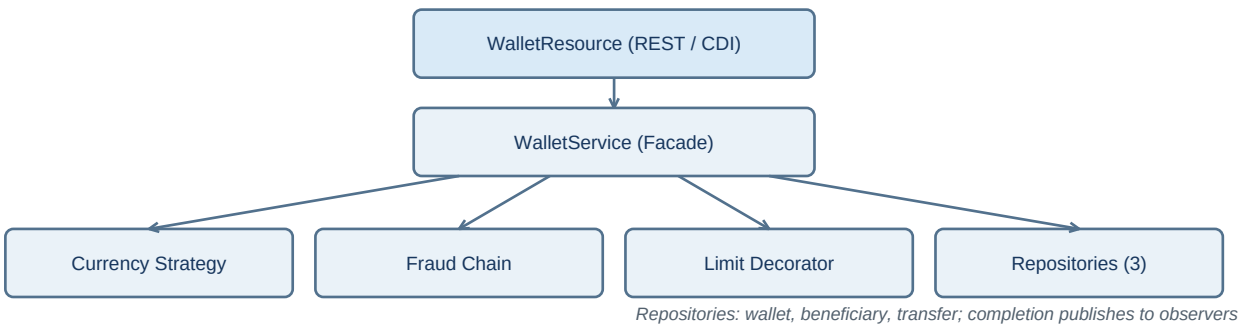
No external source repository was supplied. The baseline is the local, reproducible snapshot `baseline-local-2026-10-11` in `B9/Part1/code/digital-wallet`; the evolved snapshot is in `B9/Part2/code/digital-wallet`. The scope excludes real payment rails, databases, token storage, encryption, and network calls.

The project targets Java 17 and Quarkus 3.40.1. In the evolved app, the REST resource and CDI inject WalletService; it selects conversion, screens fraud, applies limits, verifies beneficiary ownership and balances, updates wallet balances, persists the transfer, and publishes completion. Editable architecture sources: `Part1/architecture/baseline-architecture.puml` and `Part2/diagrams/evolved-architecture.puml`.

Baseline component view



Evolved Quarkus component view



Repositories: wallet, beneficiary, transfer; completion publishes to observers

Request flow
REST Resource / CDI -> WalletService Facade -> Currency Factory / Strategy -> Fraud Check Chain -> Limit Decorator -> Wallet / Beneficiary / Transfer Repositories
WalletService -> NotificationPublisher -> EmailNotification and AuditNotification; VendorFxAdapter -> ExchangeRateProvider

Run from a Maven project root: mvn quarkus:dev to start the API or mvn test on JDK 17+. The workspace did not expose JDK or Maven, so Java execution is recorded as not run. Python and JavaScript Part 3 fixtures both returned 4/4 passed.

2. Part 1 - Pattern discovery

Candidates were accepted only when role participants and a concrete call or delegation relationship were visible. Class names, packages, comments, and LLM suggestions alone were not accepted.

Metric	Baseline result
Confirmed pattern types	7: Builder, Facade, Repository, Observer, State, Command, Value Object
Confirmed pattern instances	7 independent collaborations
Concrete roles per instance	9 pattern-specific implementation roles; shared collaborators excluded
Unique participant symbols	14; shared symbols counted once
Rejected / uncertain	0 in scope; annotations and maps were not patterns by themselves

Pattern	Instance	Roles and evidence
Builder	Transfer Record Construction	Transaction.Builder creates Transaction and rejects incomplete fields.
Facade	Wallet Transfer Application Boundary	WalletFacade coordinates WalletRepository, TransferCommand, and NotificationPublisher.
Repository	Wallet Persistence Boundary	WalletRepository find/save isolate Wallet storage.
Observer	Transaction Completion Notification	NotificationPublisher invokes EmailNotifier and AuditNotifier through TransactionObserver.
State	Transfer Lifecycle	TransactionState delegates to PendingState and CompletedState.
Command	Transfer Execution	TransferCommand.execute encapsulates debit, credit, and record construction.
Value Object	Money Amount and Currency	Immutable Money combines amount and currency with value equality.

3. Part 2 - Evolution and refactoring

CH01 - Multi-currency support

Criteria: EUR 10.00 to EUR settles unchanged, USD 10.00 to EUR settles at EUR 9.20, unsupported demo currency pairs reject, and transfer orchestration has no currency-pair conditional. The sample adapter supports only USDEUR at 0.92; it is not a live FX service. CurrencyConversionStrategy has SameCurrencyConversion and FixedRateConversion; CurrencyStrategyFactory selects one through ExchangeRateProvider; VendorFxAdapter translates VendorFxClient. Strategy, Simple Factory, and Adapter are new. The WalletRepository is retained; transfer and beneficiary repositories are added. REST request/response records are DTOs. Builder is removed because the immutable Transfer record has a fixed small field set.

CH02 - Transaction limits

Criteria: EUR 900 passes a EUR 1,000 per-transfer cap, EUR 1,001 fails, and any transfer exceeding EUR 1,000 daily use fails before save or notification. TransactionLimitPolicy supplies the variable policy; DailyLimitDecorator wraps PerTransferLimit. This replaces a growing facade conditional with composition.

CH03 - Fraud detection processing

Criteria: deny-listed beneficiaries reject, transfers above EUR 6,000 receive a velocity-review rejection, normal EUR 200 transfers continue, and the first rejection stops processing. FraudCheckChain orders DenyListCheck and VelocityCheck. The default deny list is empty and requires real configuration; velocity screening is active. This replaces monolithic fraud conditionals with Chain of Responsibility.

Final catalog: Facade; Repository (three instances); Observer; State; Command; Value Object; Strategy (currency, authentication, and limit policies); Simple Factory; Adapter; Dependency Injection; Data Transfer Object; Decorator; and Chain of Responsibility.

Snapshot	Types	Instances	Java tests	Interpretation
Baseline	7	7	not run	Wallet flow plus Money value object.
After CH01	11	14	not run	Currency, authentication, three repositories, DTOs, CDI wiring.
After CH02	12	16	not run	Limits compose instead of growing transfer flow.
After CH03	13	17	not run	Ordered fraud checks need rule governance.

Java source LOC, excluding blank lines, comment-only lines, and tests, rises from 92 baseline LOC to 180 evolved LOC. Counts are in B9/data/pattern_counts.csv and B9/data/patterns_evolution.csv. More patterns do not prove a better design; improvement is the isolation of variation from transfer orchestration.

4. Part 3 - Cross-language comparison

Currency Settlement Strategy and Transaction Completion Notification Observer are implemented in Java, Python, JavaScript, and C++. Shared fixture: convert USD 10.00 to EUR at 0.92, pass EUR 10.00 unchanged, publish t1, and receive email then audit notification.

Language	Strategy structure	Observer structure	Actual result
Java	Interface with concrete strategies and factory	Interface plus lambda observers	not run: JDK unavailable
Python	Duck-typed classes selected by factory	Callables stored and invoked	4/4 passed
JavaScript	Classes; assertion compares minor units	Functions stored and broadcast	4/4 passed
C++	Abstract base class and unique_ptr	std::function lambda observers	not run: compiler unavailable

Source LOC (Java/Python/JavaScript/C++: 23/30/15/27) and maximum function CC (2 each) are measured on the snippets, excluding comments and blank lines. Python median process time was 93.291 ms; Node median was 63.374 ms, each after one warm-up and five measured runs. These are startup-inclusive fixture timings, not steady-state throughput; raw runs are in Part3/evidence/process_timing.txt. Java and C++ were not run and their timings remain blank.

5. LLM records and human verification

Task-specific prompt summaries, concise response extracts, supplied inputs, and human evaluation notes are stored in B9/data/llm.csv. Neither is presented as a verbatim log of a separate model call. Pattern discovery output was accepted only after source review. Evolution output is design input, not test evidence. The Quarkus integration and three focused Java rule tests are authored but unexecuted. Python and JavaScript ran; Java and C++ outcomes remain not run.

6. Findings and limitations

The baseline has seven confirmed collaborations; the final snapshot has 13 pattern types across 17 instances. Evolution adds Strategy and Simple Factory for conversion/authentication selection, Adapter for vendor API isolation, CDI Dependency Injection for REST/service wiring, DTO for HTTP records, Decorator for limits, and Chain of Responsibility for fraud rules. The original WalletRepository, Facade, Observer, State, Command, and Value Object remain meaningful, with two additional repository instances. The authentication Strategy is modeled but is not wired to REST. Production still needs durable persistence, idempotency, secure authentication, a populated deny list, audit controls, rate provenance, and reconciliation.

Team contributions

Technical artefacts were prepared in this workspace. Individual names, roll numbers, and contributions were not supplied; replace this statement with each student's actual contribution and evidence link before hand-in.

Submission artefacts

Location	Contents
B9/data	patterns.csv, pattern_counts.csv, patterns_evolution.csv, cross_language.csv, llm.csv, Excel dataset
B9/Part1	baseline code, architecture source, scope and LLM evidence
B9/Part2	evolved code, three changes, diffs, diagrams, execution limitation
B9/Part3	Java, Python, JavaScript, C++ fixtures and test evidence

References

Gamma, Helm, Johnson, Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*, 1994.
Quarkus, *Creating your first application*, https://quarkus.io/guides/getting-started/.
Quarkus, *Contexts and Dependency Injection*, https://quarkus.io/guides/cdi-reference/.
Oracle, *BigDecimal Java SE 17 API*, https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/math/BigDecimal.html.